



భారతీయ సాంకేతిక విజ్ఞాన సంస్థ హైదరాబాద్
भारतीय प्रौद्योगिकी संस्थान हैदराबाद
Indian Institute of Technology Hyderabad

DSP Lab - EE2801

Experiment-4

BY

Muthyala Koushik EE24BTECH11044

2026-03-09

Department of Electrical Engineering Indian Institute of Technology Hyderabad

Contents

1 Objective	1
2 Theory	2
2.1 Direct Form FIR	2
2.2 Optimized FIR	2
2.3 Generate-Based FIR	3
3 MATLAB Implementation	3
3.1 MATLAB Code	3
3.2 Output Plots	8
4 Verilog Implementation	9
4.1 Direct Form FIR	9
4.2 Optimized FIR Implementation	13
4.3 Generate-Based FIR Implementation	17
5 Verification Using MATLAB	22
5.1 MATLAB Verification Code	22
5.2 MATLAB Direct vs Verilog Direct	23
5.3 MATLAB Optimized vs Verilog Optimized	24
5.4 MATLAB Direct vs Verilog Genvar	25
5.5 Mean Squared Error Results	25

1 Objective

FIR filter Design In MATLAB

1. using `fir1()` command, obtain filter coefficients
Sampling freq = 10000, cutoff = 1000Hz, Number of taps = 100
2. Write a MATLAB code to
 - a. read the above values into a variable
 - b. convert them into fixed point $Q(2, 14)$. Store these values into a file.
 - c. Generate 3 sinewaves of 950Hz, 1100Hz, 2000Hz and sample at 10000Hz.
 - d. Convert these values into $Q(2, 14)$ format and store them into file(s).
3. Write MATLAB code to get the output for these 3 signals using the designed filter. Plot outputs for 3 signals

In Verilog

1. read filter coefficients file and 3 signal files
2. write modules to find filter output values for each of the signal using 3 approaches discussed in the calss:
 - a. direct form
 - b. Optimized form
 - c. Usign genvar
3. Store Output results into different files.
4. Read the output values in MATLAB and make plots and verify that output plots match with those from MATLAB.

Deliverables:

1. One MATLAB file which genertes filter values and shows filter output
2. 3 Verilog files and 3 Test bench files - one each for each FIR method
3. One MATLAB file, which plots both MATLAB output signals and Verilog output signals
4. PDF file - proper write up on your approcah and related maths

2 Theory

A Finite Impulse Response (FIR) filter is a digital filter whose output depends only on a finite number of present and past input samples. FIR filters are widely used because they are always stable and can be designed to have a linear phase response. The general equation of an N -tap FIR filter is

$$y[n] = \sum_{k=0}^{N-1} h[k] x[n - k] \quad (1)$$

where $x[n]$ is the input signal, $y[n]$ is the output signal, $h[k]$ represents the filter coefficients, and N is the number of taps.

In this experiment, a low-pass FIR filter is designed to allow low-frequency components of a signal to pass while attenuating higher-frequency components. To evaluate the filter performance, sinusoidal signals of different frequencies representing the passband and stopband regions are used.

For hardware implementation, fixed-point arithmetic is preferred over floating-point arithmetic. The Q(2,14) fixed-point format is used, where the total word length is 16 bits consisting of 2 integer bits and 14 fractional bits. A floating-point value is converted into fixed-point representation as

$$x_{\text{fixed}} = \text{round}(x \times 2^{14}). \quad (2)$$

Multiplication of two Q(2,14) numbers produces a Q(4,28) result, therefore a wider accumulator is required during the summation of products.

Three FIR architectures are considered in this experiment.

2.1 Direct Form FIR

The direct form architecture implements the FIR equation directly using delay elements, multipliers, and adders. Each delayed input sample is multiplied by the corresponding coefficient and all products are summed to generate the output.

2.2 Optimized FIR

For linear-phase FIR filters, the coefficients are symmetric,

$$h[k] = h[N - 1 - k]. \quad (3)$$

Using this property, the FIR equation can be rewritten as

$$y[n] = \sum_{k=0}^{\frac{N}{2}-1} h[k] (x[n - k] + x[n - (N - 1 - k)]). \quad (4)$$

This reduces the number of multipliers by half while maintaining identical filter behavior.

2.3 Generate-Based FIR

The FIR filter can also be implemented using a modular structure where a basic processing block performs the multiply–accumulate operation for one tap. Multiple such blocks are instantiated using a generate construct to form the complete FIR filter.

The outputs from the different architectures are compared to verify that all implementations produce consistent results.

3 MATLAB Implementation

3.1 MATLAB Code

Listing 1: MATLAB Implementation for FIR Filter Design and Simulation

```
1  clc;
2  clearvars;
3  close all;
4
5  % Filter Coefficients
6  fs = 10000;
7  fc = 1000;
8  n_taps = 100;
9
10 % Normalize cutoff frequency by Nyquist frequency (Fs/2)
11 Wn = fc/(fs/2);
12
13 % Design a low-pass FIR filter
14 b = fir1(n_taps-1, Wn);
15
16 % Converting into Q(2,14)
17 b_q = round(b * 2^14);
18 b_q = max(-32768, min(32767, b_q));
19
20 % Storing the filter coefficients in a file
21 fid = fopen('filter_coeff_q214.txt', 'w');
22 fprintf(fid, '%d\n', b_q);
23 fclose(fid);
24
25 % Generate 3 sinewaves
26 N = 1000;
```

```

27 t = (0:N-1)/fs;
28
29 sinewave1 = sin(2*pi*950*t);
30 sinewave2 = sin(2*pi*1100*t);
31 sinewave3 = sin(2*pi*2000*t);
32
33 % Convert signals into Q(2,14)
34 sinewave1_q = round(sinewave1 * 2^14);
35 sinewave1_q = max(-32768, min(32767, sinewave1_q));
36
37 sinewave2_q = round(sinewave2 * 2^14);
38 sinewave2_q = max(-32768, min(32767, sinewave2_q));
39
40 sinewave3_q = round(sinewave3 * 2^14);
41 sinewave3_q = max(-32768, min(32767, sinewave3_q));
42
43 % Store signals
44 fid1 = fopen('sinewave1_q214.txt', 'w');
45 fprintf(fid1, '%d\n', sinewave1_q);
46 fclose(fid1);
47
48 fid2 = fopen('sinewave2_q214.txt', 'w');
49 fprintf(fid2, '%d\n', sinewave2_q);
50 fclose(fid2);
51
52 fid3 = fopen('sinewave3_q214.txt', 'w');
53 fprintf(fid3, '%d\n', sinewave3_q);
54 fclose(fid3);
55
56
57 % Direct FIR (integer arithmetic same as Verilog)
58 y1_dir = zeros(1,N);
59 y2_dir = zeros(1,N);
60 y3_dir = zeros(1,N);
61
62 for n = 1:N
63     acc1 = 0;
64     acc2 = 0;
65     acc3 = 0;

```

```

67     for k = 1:n_taps
68         if (n-k+1) > 0
69             acc1 = acc1 + sinewave1_q(n-k+1)*b_q(k);
70             acc2 = acc2 + sinewave2_q(n-k+1)*b_q(k);
71             acc3 = acc3 + sinewave3_q(n-k+1)*b_q(k);
72         end
73     end
74
75     y1_dir(n) = acc1;
76     y2_dir(n) = acc2;
77     y3_dir(n) = acc3;
78 end
79
80
81 % Optimized FIR (symmetric coefficients)
82 half = n_taps / 2;
83
84 for n = 1:N
85     acc1 = 0; acc2 = 0; acc3 = 0;
86
87     for k = 1:half
88
89         x1 = 0; x2 = 0;
90         if (n-k+1) > 0
91             x1 = sinewave1_q(n-k+1);
92         end
93         if (n-n_taps+k) > 0
94             x2 = sinewave1_q(n-n_taps+k);
95         end
96         acc1 = acc1 + b_q(k)*(x1+x2);
97
98
99         x1 = 0; x2 = 0;
100        if (n-k+1) > 0
101            x1 = sinewave2_q(n-k+1);
102        end
103        if (n-n_taps+k) > 0
104            x2 = sinewave2_q(n-n_taps+k);
105        end
106        acc2 = acc2 + b_q(k)*(x1+x2);

```

```

107
108
109     x1 = 0; x2 = 0;
110     if (n-k+1) > 0
111         x1 = sinewave3_q(n-k+1);
112     end
113     if (n-n_taps+k) > 0
114         x2 = sinewave3_q(n-n_taps+k);
115     end
116     acc3 = acc3 + b_q(k)*(x1+x2);
117
118 end
119
120 y1_opt(n) = acc1;
121 y2_opt(n) = acc2;
122 y3_opt(n) = acc3;
123 end
124
125 % Save outputs for verification
126 fid = fopen('direct_950.txt','w'); fprintf(fid,'%d\n',y1_dir); fclose(fid);
127 fid = fopen('direct_1100.txt','w'); fprintf(fid,'%d\n',y2_dir); fclose(fid);
128 fid = fopen('direct_2000.txt','w'); fprintf(fid,'%d\n',y3_dir); fclose(fid);
129
130 fid = fopen('optimized_950.txt','w'); fprintf(fid,'%d\n',y1_opt); fclose(fid);
131 fid = fopen('optimized_1100.txt','w'); fprintf(fid,'%d\n',y2_opt); fclose(fid);
132 fid = fopen('optimized_2000.txt','w'); fprintf(fid,'%d\n',y3_opt); fclose(fid);
133
134
135
136 % Convert to floating point for plotting
137 output1_dir = y1_dir / 2^28;
138 output2_dir = y2_dir / 2^28;
139 output3_dir = y3_dir / 2^28;
140
141 output1_opt = y1_opt / 2^28;
142 output2_opt = y2_opt / 2^28;
143 output3_opt = y3_opt / 2^28;
144
145
146 % Plot outputs

```

```

147
148 figure;
149
150 subplot(3,1,1)
151 plot(t,output1_dir)
152 hold on
153 plot(t,output1_opt, '—')
154 title('950 Hz Output')
155 legend('Direct', 'Optimized')
156 xlabel('Time (s)')
157 ylabel('Amplitude')
158 xlim([0 0.05])
159
160 subplot(3,1,2)
161 plot(t,output2_dir)
162 hold on
163 plot(t,output2_opt, '—')
164 title('1100 Hz Output')
165 legend('Direct', 'Optimized')
166 xlabel('Time (s)')
167 ylabel('Amplitude')
168 xlim([0 0.05])
169
170 subplot(3,1,3)
171 plot(t,output3_dir)
172 hold on
173 plot(t,output3_opt, '—')
174 title('2000 Hz Output')
175 legend('Direct', 'Optimized')
176 xlabel('Time (s)')
177 ylabel('Amplitude')
178 xlim([0 0.05])

```

Explanation:

The MATLAB program is used to design and simulate the FIR filter before hardware implementation. First, the filter parameters such as sampling frequency, cutoff frequency, and number of taps are defined. Using these parameters, the FIR filter coefficients are generated. Since the hardware implementation uses fixed-point arithmetic, the coefficients are converted into Q(2,14) format.

Three sinusoidal input signals with different frequencies are generated to test the behavior of the filter in the passband and stopband regions. These signals are also converted to fixed-point representation so that the same values can be used in the Verilog simulation.

The FIR filtering operation is then performed using two approaches: direct form and optimized form. In the direct form implementation, the convolution between the input signal and filter coefficients is computed directly. In the optimized implementation, the symmetry of the filter coefficients is used to reduce the number of multiplications.

The resulting outputs are stored and later used for verification with the Verilog implementations.

3.2 Output Plots

The filtered outputs are plotted to observe the behavior of the filter for different input frequencies.

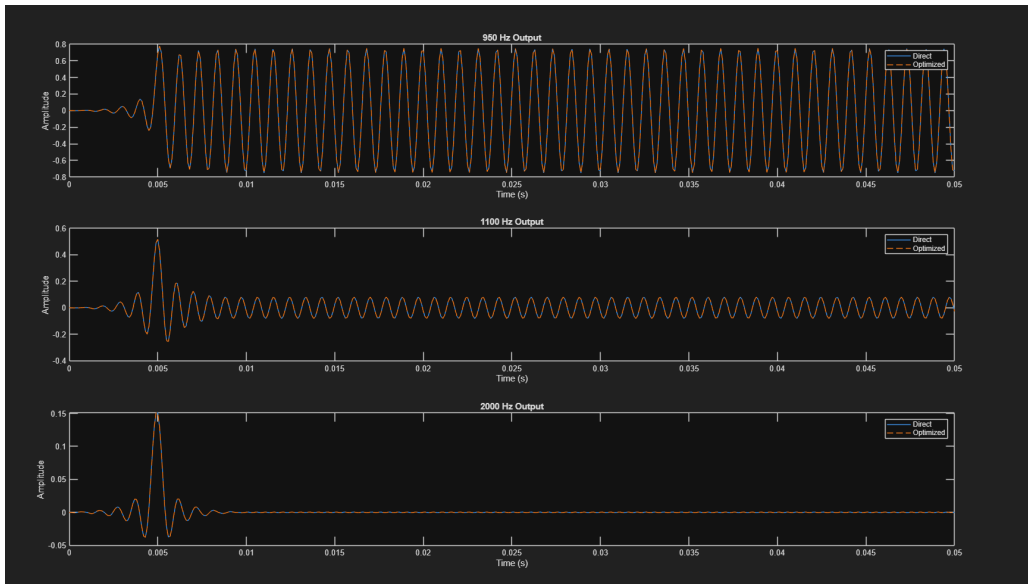


Figure 1: Filtered outputs for input signals of 950 Hz, 1100 Hz, and 2000 Hz.

4 Verilog Implementation

4.1 Direct Form FIR

Listing 2: Direct Form FIR Module

```

1  module fir_direct(
2      input  wire clk,
3      input  wire rst,
4      input  wire signed [15:0] x_in,
5      input  wire signed [15:0] h [0:99],
6      output reg  signed [39:0] y_out
7  );
8
9  parameter TAPS = 100;
10
11 // Accumulator temp register
12 reg signed [39:0] acc;
13
14 // Delay line
15 reg signed [15:0] x_delay [0:TAPS-1];
16
17 integer i;
18
19 // Shift register (delay line)
20 always @(posedge clk or posedge rst) begin
21     if(rst) begin
22         for(i = 0; i < TAPS; i = i + 1)
23             x_delay[i] <= 16'sd0;
24     end
25     else begin
26         for(i = TAPS-1; i > 0; i = i - 1)
27             x_delay[i] <= x_delay[i-1];
28
29         x_delay[0] <= x_in;
30     end
31 end
32
33 // FIR computation
34 always @(*) begin
    
```

```

35     acc = 40'sd0;
36     for(i = 0; i < TAPS; i = i + 1)
37         acc = acc + (x_delay[i] * h[i]);
38     y_out = acc;
39 end
40
41 endmodule

```

Listing 3: Testbench for Direct Form FIR

```

1  `timescale 1ns / 1ps
2
3  module tb_direct;
4  reg clk;
5  reg rst;
6  reg signed [15:0] x_in;
7  wire signed [39:0] y_out;
8
9  //signal memory
10 reg signed [15:0] x_signal1 [0:999];
11 reg signed [15:0] x_signal2 [0:999];
12 reg signed [15:0] x_signal3 [0:999];
13
14 //coefficients
15 reg signed [15:0] h [0:99];
16
17 integer i;
18 integer fid_1, fid_2, fid_3;
19 integer dummy_1, dummy_2, dummy_3;
20 integer fid_out1, fid_out2, fid_out3;
21 integer fid_coeff;
22
23 fir_direct uut (
24     .clk(clk),
25     .rst(rst),
26     .x_in(x_in),
27     .h(h),
28     .y_out(y_out)
29 );
30
31 // Clock generation

```

```

32 initial clk = 0;
33 always #5 clk = ~clk; // 100 MHz clock
34
35 // Simulation
36 initial begin
37
38     // Load coefficients
39     fid_coeff = $fopen("filter_coeff_q214.txt", "r");
40     for(i = 0; i < 100; i = i + 1) begin
41         dummy_1 = $fscanf(fid_coeff, "%d", h[i]);
42     end
43     $fclose(fid_coeff);
44
45     // Load input signals
46     fid_1 = $fopen("sinewave1_q214.txt", "r");
47     for(i = 0; i < 1000; i = i + 1) begin
48         dummy_1 = $fscanf(fid_1, "%d", x_signal1[i]);
49     end
50     $fclose(fid_1);
51
52     fid_2 = $fopen("sinewave2_q214.txt", "r");
53     for(i = 0; i < 1000; i = i + 1) begin
54         dummy_2 = $fscanf(fid_2, "%d", x_signal2[i]);
55     end
56     $fclose(fid_2);
57
58     fid_3 = $fopen("sinewave3_q214.txt", "r");
59     for(i = 0; i < 1000; i = i + 1) begin
60         dummy_3 = $fscanf(fid_3, "%d", x_signal3[i]);
61     end
62     $fclose(fid_3);
63
64     // Open output files
65     fid_out1 = $fopen("output_direct1_q214.txt", "w");
66     fid_out2 = $fopen("output_direct2_q214.txt", "w");
67     fid_out3 = $fopen("output_direct3_q214.txt", "w");
68
69     // Reset
70     rst = 1;
71     #20;

```

```
72     rst = 0;
73
74     // Process sinewave1
75     for(i = 0; i < 1000; i = i + 1) begin
76         x_in = x_signal1[i];
77         #10; // Wait for one clock cycle
78         $fwrite(fid_out1, "%d\n", y_out);
79     end
80
81     // Reset for next signal
82     rst = 1;
83     #20;
84     rst = 0;
85
86     // Process sinewave2
87     for(i = 0; i < 1000; i = i + 1) begin
88         x_in = x_signal2[i];
89         #10; // Wait for one clock cycle
90         $fwrite(fid_out2, "%d\n", y_out);
91     end
92
93     // Reset for next signal
94     rst = 1;
95     #20;
96     rst = 0;
97
98     // Process sinewave3
99     for(i = 0; i < 1000; i = i + 1) begin
100         x_in = x_signal3[i];
101         #10; // Wait for one clock cycle
102         $fwrite(fid_out3, "%d\n", y_out);
103     end
104
105     // Close output files and finish simulation
106     $fclose(fid_out1);
107     $fclose(fid_out2);
108     $fclose(fid_out3);
109     $finish;
110 end
111
```

```
112
113 endmodule
```

Explanation:

The direct form FIR implementation follows the standard convolution equation of the FIR filter. A delay line is used to store the previous input samples, where each clock cycle shifts the input through the delay registers. Each delayed sample is multiplied with the corresponding filter coefficient, and the resulting products are accumulated to generate the output signal.

The module contains a delay register array for storing past input samples and an accumulator that performs the sum of all multiplication results. The output is obtained by summing the products of the delayed inputs and the filter coefficients.

The testbench reads the filter coefficients and input signals from text files generated by MATLAB. The input samples are applied sequentially to the FIR module, and the resulting outputs are written to output files. These outputs are later used for comparison with MATLAB results.

4.2 Optimized FIR Implementation

Listing 4: Optimized FIR Module Using Symmetry

```
1 module fir_optimized(
2     input wire clk,
3     input wire rst,
4     input wire signed [15:0] x_in,
5     input wire signed [15:0] h [0:99],
6     output reg signed [39:0] y_out
7 );
8
9 parameter TAPS = 100;
10
11 // Accumulator temp register
12 reg signed [39:0] acc;
13
14 // Symmetric sum temp register
15 reg signed [16:0] sym_sum;
16
17 // Delay line
18 reg signed [15:0] x_delay [0:TAPS-1];
19
20 integer i;
21
22 // Shift register (delay line)
```

```

23 always @(posedge clk or posedge rst) begin
24     if(rst) begin
25         for(i = 0; i < TAPS; i = i + 1)
26             x_delay[i] <= 16'sd0;
27     end
28     else begin
29         for(i = TAPS-1; i > 0; i = i - 1)
30             x_delay[i] <= x_delay[i-1];
31     end
32     x_delay[0] <= x_in;
33 end
34 end
35
36 // Optimized FIR computation using symmetry: h[k] = h[N-1-k]
37 always @(*) begin
38     acc = 40'sd0;
39     // Pair symmetric coefficients: h[k] * (x[n-k] + x[n-(N-1-k)])
40     for(i = 0; i < TAPS/2; i = i + 1) begin
41         sym_sum = {x_delay[i][15], x_delay[i]} + {x_delay[TAPS-1-i][15],
42             x_delay[TAPS-1-i]};
43         acc = acc + (h[i] * sym_sum);
44     end
45     y_out = acc;
46 end
47 endmodule

```

Listing 5: Testbench for Optimized FIR

```

1 `timescale 1ns / 1ps
2
3 module tb_optimized;
4     reg clk;
5     reg rst;
6     reg signed [15:0] x_in;
7     wire signed [39:0] y_out;
8
9     //signal memory
10    reg signed [15:0] x_signal1 [0:999];
11    reg signed [15:0] x_signal2 [0:999];
12    reg signed [15:0] x_signal3 [0:999];

```

```

13
14 //coefficients
15 reg signed [15:0] h [0:99];
16
17 integer i;
18 integer fid_1, fid_2, fid_3;
19 integer dummy_1, dummy_2, dummy_3;
20 integer fid_out1, fid_out2, fid_out3;
21 integer fid_coeff;
22
23 fir_optimized uut (
24     .clk(clk),
25     .rst(rst),
26     .x_in(x_in),
27     .h(h),
28     .y_out(y_out)
29 );
30
31 // Clock generation
32 initial clk = 0;
33 always #5 clk = ~clk; // 100 MHz clock
34
35 // Simulation
36 initial begin
37
38     // Load coefficients
39     fid_coeff = $fopen("filter_coeff_q214.txt", "r");
40     for(i = 0; i < 100; i = i + 1) begin
41         dummy_1 = $fscanf(fid_coeff, "%d", h[i]);
42     end
43     $fclose(fid_coeff);
44
45     // Load input signals
46     fid_1 = $fopen("sinewave1_q214.txt", "r");
47     for(i = 0; i < 1000; i = i + 1) begin
48         dummy_1 = $fscanf(fid_1, "%d", x_signal1[i]);
49     end
50     $fclose(fid_1);
51
52     fid_2 = $fopen("sinewave2_q214.txt", "r");

```

```
53     for(i = 0; i < 1000; i = i + 1) begin
54         dummy_2 = $fscanf(fid_2,"%d",x_signal2[i]);
55     end
56     $fclose(fid_2);
57
58     fid_3 = $fopen("sinewave3_q214.txt","r");
59     for(i = 0; i < 1000; i = i + 1) begin
60         dummy_3 = $fscanf(fid_3,"%d",x_signal3[i]);
61     end
62     $fclose(fid_3);
63
64     // Open output files
65     fid_out1 = $fopen("output_optimized1_q214.txt","w");
66     fid_out2 = $fopen("output_optimized2_q214.txt","w");
67     fid_out3 = $fopen("output_optimized3_q214.txt","w");
68
69     // Reset
70     rst = 1;
71     #20;
72     rst = 0;
73
74     // Process sinewave1
75     for(i = 0; i < 1000; i = i + 1) begin
76         x_in = x_signal1[i];
77         #10; // Wait for one clock cycle
78         $fwrite(fid_out1, "%d\n", y_out);
79     end
80
81     // Reset for next signal
82     rst = 1;
83     #20;
84     rst = 0;
85
86     // Process sinewave2
87     for(i = 0; i < 1000; i = i + 1) begin
88         x_in = x_signal2[i];
89         #10; // Wait for one clock cycle
90         $fwrite(fid_out2, "%d\n", y_out);
91     end
92
```

```

93 // Reset for next signal
94 rst = 1;
95 #20;
96 rst = 0;
97
98 // Process sinewave3
99 for(i = 0; i < 1000; i = i + 1) begin
100     x_in = x_signal3[i];
101     #10; // Wait for one clock cycle
102     $fwrite(fid_out3, "%d\n", y_out);
103 end
104
105 // Close output files and finish simulation
106 $fclose(fid_out1);
107 $fclose(fid_out2);
108 $fclose(fid_out3);
109 $finish;
110 end
111
112
113 endmodule

```

Explanation:

The optimized FIR implementation exploits the symmetry property of linear-phase FIR filters. For such filters, the coefficients satisfy the relation $h[k] = h[N - 1 - k]$. Using this property, pairs of symmetric input samples are added before multiplication with the corresponding coefficient.

This reduces the number of multipliers required in the implementation by approximately half, improving hardware efficiency while producing the same output as the direct form implementation.

The testbench structure is similar to that of the direct form implementation. It loads the input signals and coefficients from files, feeds the samples to the FIR module, and stores the resulting outputs for verification.

4.3 Generate-Based FIR Implementation

Listing 6: FIR Implementation Using Generate Construct

```

1 module fir_genvar(
2     input wire clk,
3     input wire rst,
4     input wire signed [15:0] x_in,
5     input wire signed [15:0] h [0:99],

```

```
6      output reg signed [39:0] y_out
7  );
8
9  parameter TAPS = 100;
10
11 // Delay line
12 reg signed [15:0] x_delay [0:TAPS-1];
13
14 integer i;
15
16 // Shift register (delay line)
17 always @(posedge clk or posedge rst) begin
18     if(rst) begin
19         for(i = 0; i < TAPS; i = i + 1)
20             x_delay[i] <= 16'sd0;
21     end
22     else begin
23         for(i = TAPS-1; i > 0; i = i - 1)
24             x_delay[i] <= x_delay[i-1];
25
26         x_delay[0] <= x_in;
27     end
28 end
29
30 // FIR computation using generate and FPB modules
31 wire signed [39:0] sum [0:TAPS];
32 assign sum[0] = 40'sd0;
33
34 genvar k;
35 generate
36     for(k = 0; k < TAPS; k = k + 1) begin : fir_tap
37         FPB fpb_inst(
38             .a_in(x_delay[k]),
39             .b_in(sum[k]),
40             .h_i(h[k]),
41             .a_out(),
42             .b_out(sum[k+1])
43         );
44     end
45 endgenerate
```

```

46
47 // Output assignment
48 always @(*) begin
49     y_out = sum[TAPS];
50 end
51
52 endmodule
53
54 module FPB(
55     input signed [15:0] a_in,
56     input signed [39:0] b_in,
57     input signed [15:0] h_i,
58     output signed [15:0] a_out,
59     output signed [39:0] b_out
60 );
61
62 assign a_out = a_in;
63
64 assign b_out = b_in + {{8{(a_in*h_i)[31]}}, (a_in*h_i)};
65
66 endmodule
    
```

Listing 7: Testbench for Generate-Based FIR

```

1 `timescale 1ns / 1ps
2
3 module tb_genvar;
4     reg clk;
5     reg rst;
6     reg signed [15:0] x_in;
7     wire signed [39:0] y_out;
8
9     //signal memory
10    reg signed [15:0] x_signal1 [0:999];
11    reg signed [15:0] x_signal2 [0:999];
12    reg signed [15:0] x_signal3 [0:999];
13
14    //coefficients
15    reg signed [15:0] h [0:99];
16
17    integer i;
    
```

```

18 integer fid_1, fid_2, fid_3;
19 integer dummy_1, dummy_2, dummy_3;
20 integer fid_out1, fid_out2, fid_out3;
21 integer fid_coeff;
22
23 fir_genvar uut (
24     .clk(clk),
25     .rst(rst),
26     .x_in(x_in),
27     .h(h),
28     .y_out(y_out)
29 );
30
31 // Clock generation
32 initial clk = 0;
33 always #5 clk = ~clk; // 100 MHz clock
34
35 // Simulation
36 initial begin
37
38     // Load coefficients
39     fid_coeff = $fopen("filter_coeff_q214.txt", "r");
40     for(i = 0; i < 100; i = i + 1) begin
41         dummy_1 = $fscanf(fid_coeff, "%d", h[i]);
42     end
43     $fclose(fid_coeff);
44
45     // Load input signals
46     fid_1 = $fopen("sinewave1_q214.txt", "r");
47     for(i = 0; i < 1000; i = i + 1) begin
48         dummy_1 = $fscanf(fid_1, "%d", x_signal1[i]);
49     end
50     $fclose(fid_1);
51
52     fid_2 = $fopen("sinewave2_q214.txt", "r");
53     for(i = 0; i < 1000; i = i + 1) begin
54         dummy_2 = $fscanf(fid_2, "%d", x_signal2[i]);
55     end
56     $fclose(fid_2);
57

```

```

58     fid_3 = $fopen("sinewave3_q214.txt","r");
59     for(i = 0; i < 1000; i = i + 1) begin
60         dummy_3 = $fscanf(fid_3,"%d",x_signal3[i]);
61     end
62     $fclose(fid_3);
63
64     // Open output files
65     fid_out1 = $fopen("output_genvar1_q214.txt","w");
66     fid_out2 = $fopen("output_genvar2_q214.txt","w");
67     fid_out3 = $fopen("output_genvar3_q214.txt","w");
68
69     // Reset
70     rst = 1;
71     #20;
72     rst = 0;
73
74     // Process sinewave1
75     for(i = 0; i < 1000; i = i + 1) begin
76         x_in = x_signal1[i];
77         #10; // Wait for one clock cycle
78         $fwrite(fid_out1, "%d\n", y_out);
79     end
80
81     // Reset for next signal
82     rst = 1;
83     #20;
84     rst = 0;
85
86     // Process sinewave2
87     for(i = 0; i < 1000; i = i + 1) begin
88         x_in = x_signal2[i];
89         #10; // Wait for one clock cycle
90         $fwrite(fid_out2, "%d\n", y_out);
91     end
92
93     // Reset for next signal
94     rst = 1;
95     #20;
96     rst = 0;
97

```

```

98 // Process sinewave3
99 for(i = 0; i < 1000; i = i + 1) begin
100     x_in = x_signal3[i];
101     #10; // Wait for one clock cycle
102     $fwrite(fid_out3, "%d\n", y_out);
103 end
104
105 // Close output files and finish simulation
106 $fclose(fid_out1);
107 $fclose(fid_out2);
108 $fclose(fid_out3);
109 $finish;
110 end
111
112
113 endmodule

```

Explanation:

The generate-based implementation uses the Verilog generate construct to instantiate multiple filter processing blocks automatically. Each processing block performs the multiply–accumulate operation corresponding to one filter tap.

By using a genvar loop, the design becomes modular and scalable, allowing the FIR filter structure to be created systematically without manually writing code for each tap. This approach improves code readability and simplifies the implementation of filters with a large number of taps.

The testbench reads the same input signals and coefficients used in the previous implementations and records the outputs generated by the FIR module for comparison with MATLAB results.

5 Verification Using MATLAB

5.1 MATLAB Verification Code

Listing 8: MATLAB Code for Verification of Verilog Outputs

```

1 % Paste the MATLAB verification script here

```

Explanation:

The MATLAB verification script is used to validate the correctness of the Verilog implementations. The outputs generated from MATLAB simulations and the outputs obtained from the Verilog simulations are read from text files.

Since the hardware implementation uses fixed-point arithmetic, the integer outputs are converted back to floating-point values by dividing them by 2^{28} . This restores the numerical scale of the filtered signals.

The MATLAB script then compares the outputs from MATLAB and Verilog implementations for the three architectures: Direct FIR, Optimized FIR, and Generate-based FIR. The comparison is performed using the Mean Squared Error (MSE) metric, which measures the average squared difference between two signals. If the MSE is zero, it indicates that the outputs from MATLAB and Verilog match exactly.

Finally, MATLAB plots the signals from both implementations for visual verification.

5.2 MATLAB Direct vs Verilog Direct

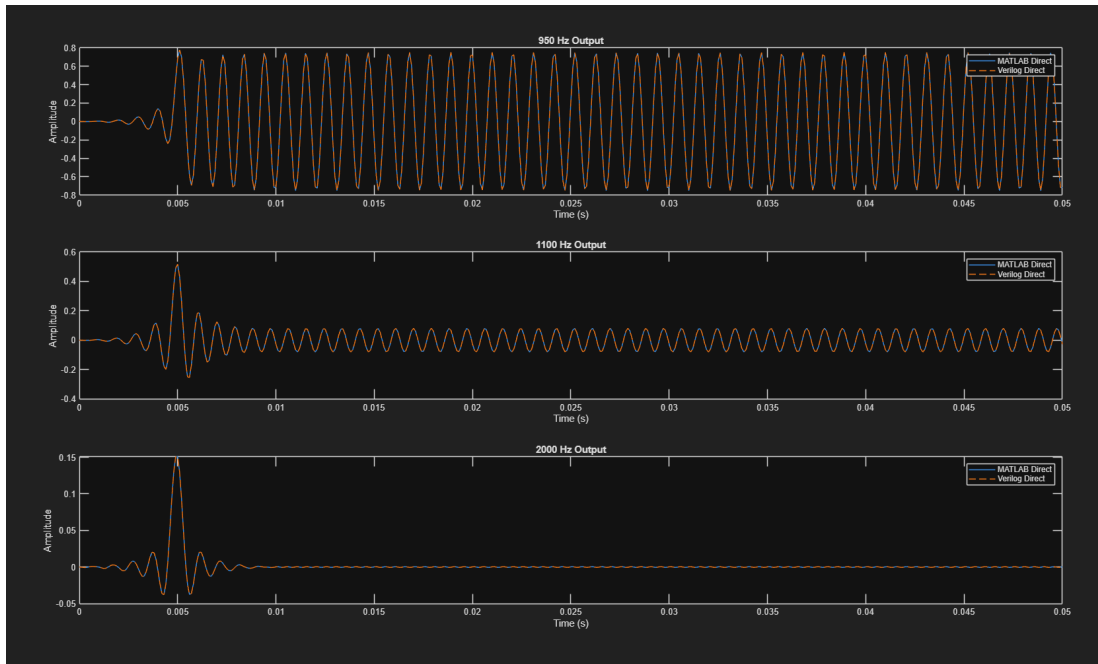


Figure 2: Comparison between MATLAB Direct FIR output and Verilog Direct FIR output for input frequencies of 950 Hz, 1100 Hz, and 2000 Hz.

5.3 MATLAB Optimized vs Verilog Optimized

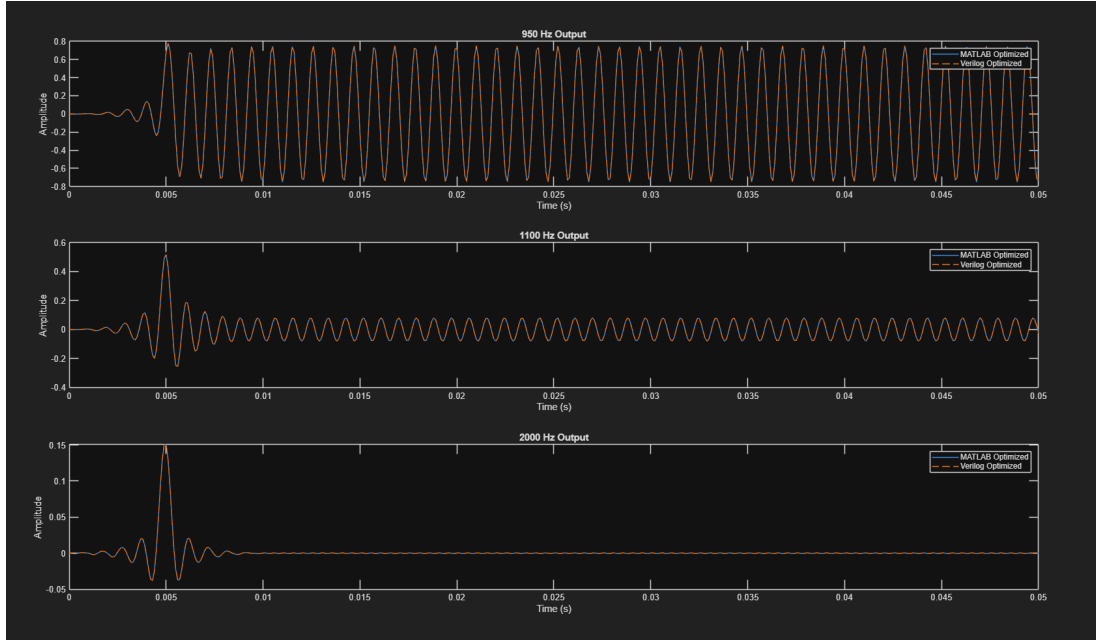


Figure 3: Comparison between MATLAB Optimized FIR output and Verilog Optimized FIR output for input frequencies of 950 Hz, 1100 Hz, and 2000 Hz.

5.4 MATLAB Direct vs Verilog Genvar

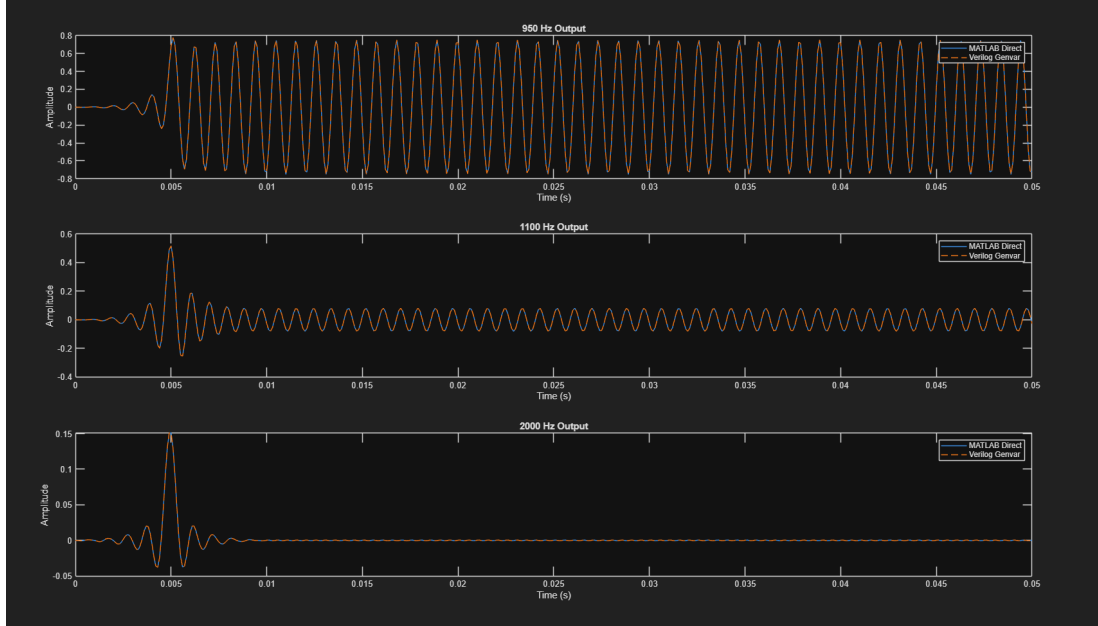


Figure 4: Comparison between MATLAB Direct FIR output and Verilog Generate-based FIR output for input frequencies of 950 Hz, 1100 Hz, and 2000 Hz.

5.5 Mean Squared Error Results

The Mean Squared Error (MSE) values between MATLAB and Verilog outputs are listed below:

MSE: MATLAB Direct vs Verilog Direct

950 Hz: 0.000000e+00

1100 Hz: 0.000000e+00

2000 Hz: 0.000000e+00

MSE: MATLAB Optimized vs Verilog Optimized

950 Hz: 0.000000e+00

1100 Hz: 0.000000e+00

2000 Hz: 0.000000e+00

MSE: MATLAB Direct vs Verilog Genvar

950 Hz: 0.000000e+00

1100 Hz: 0.000000e+00

2000 Hz: 0.000000e+00

The zero MSE values indicate that the outputs obtained from MATLAB and the Verilog implementations are identical, confirming the correctness of the FIR filter design and hardware realization.